

MENAGERIE FINAL MASTER PLAN

Single-VM | 6-Agent Swarm | Predictive Wall Clock | Full Code

Digital Principles Corp | March 28, 2026 | VERSION FINAL

Complete spec. Replaces ANVIL fleet (3x M1 Macs) with one Azure D4s_v5 VM 24/7. Architecture, wiring, speed, 15 errors, 6 agents, wall clock, all code, CLAUDE.md. Drop into Claude Code and execute.

TABLE OF CONTENTS

#	Section
01	EXECUTIVE SUMMARY
02	WHY SINGLE VM
03	ARCHITECTURE + ENDPOINTS
04	WIRING DIAGRAMS (4)
05	WORLD-CLASS SPEED (3 CONFIGS)
06	HTTP vs API vs SDK
07	ERROR PREVENTION (15)
08	6-AGENT SWARM + PROMPTS
09	PREDICTIVE WALL CLOCK
10	SETUP SCRIPT
11	CLAUDE.md DROP-IN
12	NGINX CONFIG
13	FASTAPI SKELETON
14	SYSTEMD + BACKUP
15	COST ANALYSIS
16	TIMELINE (3 PHASES)
17	POST-DEPLOY CHECKLIST (18)

01 EXECUTIVE SUMMARY

Single Azure D4s_v5 (4 vCPU, 16GB), Ubuntu 24.04, always-on. Replaces ANVIL fleet. Nginx HTTP/2 + FastAPI + PG16 pgvector + Redis on localhost. Blob/AI Search/OpenAI via SDK. Tauri speaks HTTPS.

Decision	Choice	Rationale
Compute	D4s_v5 VM	Always hot, localhost DB <5ms
ANVIL	RETIRED x3	VM faster, cheaper, one target
API	FastAPI uvicorn x4	Async, SSE, auto-restart
DB	PG16 localhost	pgvector, PgBouncer, zero-latency
Cache	Redis localhost	<1ms, 2GB LRU
Files	Azure Blob	SAS direct upload
Search	AI Search	Managed, semantic+vector
AI	Azure OpenAI	SSE streaming, Anthropic fallback
Proxy	Nginx HTTP/2+gzip	70-80% compression
Frontend	Tauri	Native, single HTTP client
Deploy	ssh+git pull+systemctl	10 seconds

Pricing	Monthly	Annual
PAYG	\$296	\$3,552
1yr reserved	\$241	\$2,892
3yr reserved	\$211	\$2,532

02 WHY SINGLE VM

Pattern	Pros	Cons	Verdict
P1: Direct SDK	Fast build	Secrets on client, no auth	REJECTED
P2: Functions	Auto-scale	1-3s cold starts, \$150/mo fix	REJECTED
P3: Single VM	Always hot, <5ms DB, cheapest	No auto-scale (OK at scale)	SELECTED

03 ARCHITECTURE

```
=====
MENAGERIE VM STACK (D4s_v5)
=====
TAURI ---- HTTPS (HTTP/2, TLS1.3, keep-alive, gzip)
|
[NGINX] -- reverse proxy, compression, rate limit
|
[FASTAPI]- uvicorn x4, systemd Restart=always
|
+-----+-----+-----+-----+
| | | | |
[PG16] [REDIS] [BLOB] [SEARCH][OPENAI]
local local SDK SDK SSE
51tbl <1ms SAS 50-200ms ~300ms
pgvec 2GB direct semantic TTFT
=====
```

Endpoint	Purpose	Zone	Latency
POST /api/auth/token	JWT	GREEN	<10ms
GET /api/health	Health	GREEN	<1ms
POST /api/crud/{t}	Create	YELLOW	<10ms
GET /api/crud/{t}/{id}	Read	GREEN	<5ms
PUT /api/crud/{t}/{id}	Update	YELLOW	<10ms
DELETE /api/crud/{t}/{id}	Delete	RED	<10ms
POST /api/upload/sas	SAS token	GREEN	<5ms
POST /api/upload/confirm	Confirm+vec	YELLOW	<10ms
POST /api/search	Search	GREEN	50-200ms
POST /api/chat	AI SSE	GREEN	~300ms
GET /api/vectors/{id}	Embeddings	GREEN	<20ms
POST /api/batch/vectorize	Batch async	YELLOW	Async
GET /api/jobs/{id}	Job status	GREEN	<5ms

04 WIRING DIAGRAMS

A: CRUD Flow

```
TAURI --POST /api/crud--> [NGINX] --> [FASTAPI]
1. Pydantic validate 2. Redis: HIT=1ms, MISS=PG 5ms
3. Write PG 4. Invalidate cache 5. gzip response
<-- 200 {id:42} -- TOTAL: 10-20ms
```

B: SAS Upload

```
TAURI --POST /upload/sas--> VM generates SAS (2ms)
TAURI --PUT bytes-----> AZURE BLOB (direct, VM bypassed)
TAURI --POST /confirm-----> VM: metadata->PG, async vectorize
User sees: instant. File never touches VM.
```

C: AI Chat SSE

```
TAURI --POST /chat--> VM builds msgs, fetches context
VM --stream request--> AZURE OPENAI
VM <--chunks-- OPENAI | TAURI <--SSE tokens-- VM
First token ~300ms. No WebSocket. Stateless.
```

D: Vectorization (Background)

```
[CONFIRM] -> [WORKER]: fetch blob -> extract text ->
chunk (512tok/50overlap) -> embed (3072d) ->
pgvector INSERT -> AI Search push -> job=complete
On 429: Anthropic fallback. Duration: 5-30s/doc.
```

05 WORLD-CLASS SPEED

Three config changes. 80-120ms -> 5-15ms. No code changes.

- 1. HTTP/2: listen 443 ssl http2; -- one connection, all requests multiplexed
- 2. Connection pool: lazy_static request::Client, http2_prior_knowledge, pool_max_idle=10
- 3. Compression: gzip on; gzip_comp_level 6; gzip_min_length 256; -- 70-80% smaller

Op	Before	After	How
CRUD cached	80-120ms	<1ms	Redis local
CRUD DB	80-120ms	<5ms	PG local
Upload 10MB	5-10s	<2s	SAS direct
Search	300-500ms	50-200ms	No hop
AI 1st token	1-3s	~300ms	SSE
Next HTTP	80-120ms	5-15ms	H2+pool+gzip

06 HTTP vs API vs SDK

API call IS an HTTP call. Same wire. The real distinction: REST (raw HTTP, Tauri->VM) vs SDK (library wrapping HTTP, VM->Azure). SSE = one-way stream over HTTP for AI tokens. No WebSocket anywhere.

```
TAURI ---REST/HTTP---> VM (FastAPI)
|-- azure-storage-blob --> Blob
|-- openai SDK -----> OpenAI
|-- asyncpg (direct) ----> PG (localhost)
|-- redis-py (direct) --> Redis (localhost)
Tauri: one address, HTTP, done.
```


07 ERROR PREVENTION (15)

ID	Failure	Risk	Prevention
E01	VM boot	HIGH	systemd Restart=always WantedBy=multi-user
E02	PG exhaust	HIGH	PgBouncer txn mode, asyncpg pool 5-20
E03	Disk full	HIGH	Never store files. logrotate 7d. Alert 80%
E04	SSL expire	MEDIUM	Certbot auto-renew cron
E05	API crash	MEDIUM	Restart=always, 4 workers, UptimeRobot
E06	Redis OOM	LOW	2gb max, allkeys-lru
E07	CORS	HIGH	Nginx header tauri://localhost
E08	Upload timeout	MEDIUM	SAS bypasses VM. 10m body limit
E09	OpenAI 429	MEDIUM	tenacity 3x backoff, Anthropic fallback
E10	SSH brute	HIGH	NSG IP-lock, fail2ban 3/1hr, key-only
E11	PG data loss	CRITICAL	pg_dump nightly->Blob, weekly snapshots
E12	Worker fail	MEDIUM	Job status table, stale >10m = alert
E13	DNS delay	LOW	TTL=300s, allow IP+domain
E14	Deps	LOW	venv, pin requirements.txt
E15	SDK token	MEDIUM	DefaultAzureCredential, auto-refresh

08 6-AGENT SWARM

A G	Role	Scope	Zone	Time
1	INFRA	VM, Nginx, systemd, TLS, firewall	GREEN	30m
2	DB	PG16, pgvector, schema, PgBouncer, Redis, backup	YELLOW	30m
3	API	FastAPI 13 endpoints, Redis, SSE, Pydantic	YELLOW	60m
4	STORAGE	Blob, SAS, AI Search index, queries	GREEN	30m
5	AI	OpenAI stream, embeddings, chunker, RAG	YELLOW	30m
6	SECURITY	NSG, fail2ban, SSH, identity, Monitor	RED	30m

AG-1 FIRST -> [VM READY] -> AG-2,3,4,5,6 ALL PARALLEL -> [INTEGRATE] -> [LIVE]

Universal Rules

```
# ALL AGENTS: RED=human. YELLOW=tests. GREEN=auto.  
# Time: 1/5/10/30/60m. >2hr=STOP.  
# STATUS.md after each step. BLOCKED=stop.  
# Never cross file ownership. Secrets via .env.
```

AG-1 INFRA

```
FIRST. 30m GREEN.  
1.az vm create D4s_v5 Ubuntu2404  
2.apt install nginx pg16 pgvector redis python3.12 certbot fail2ban ufw  
3.Nginx: HTTP/2 TLS1.3 gzip6 proxy 8000  
4.systemd template, logrotate 7d, UFW 443+22  
5.VERIFY: curl -I -> H2 200 gzip  
6.STATUS: INFRA=COMPLETE
```

AG-2 DB

```
WAIT INFRA. 30m YELLOW.  
1.CREATE DATABASE+USER+EXTENSION vector  
2.Load 51-table schema  
3.PG: max_conn=200 shared_buf=4GB  
4.PgBouncer txn mode max=400  
5.Redis: 2gb allkeys-lru  
6.Backup: pg_dump|gzip|az blob, cron 03:00  
7.VERIFY: 51+ tables, redis PONG  
8.STATUS: DATABASE=COMPLETE
```

AG-3 API

```
WAIT INFRA. 60m YELLOW.  
1.venv, pip install all deps  
2.App: main.py routers/ services/ models/  
3.asyncpg pool 5-20 via PgBouncer:6432  
4.Redis cache 300s TTL  
5.SSE streaming /api/chat  
6.Pydantic + structured errors + logging MW  
7.Import AG-4 storage + AG-5 AI  
8.systemd 4 workers  
9.VERIFY: all 13 endpoints
```

```
10.STATUS: API=COMPLETE
```

AG-4 STORAGE

```
WAIT INFRA. 30m GREEN.  
1.services/storage.py  
2.BlobServiceClient DefaultAzureCredential  
3.SAS: 5min write-only per-blob  
4.Upload confirm: metadata->PG  
5.AI Search index: id,title,content,vector(3072)  
6.Push+search with filters/facets/vector  
7.VERIFY: SAS gen, upload, search  
8.STATUS: STORAGE=COMPLETE
```

AG-5 AI

```
WAIT INFRA. 30m YELLOW.  
1.services/ai.py  
2.AzureOpenAI DefaultAzureCredential  
3.Stream chat: gpt-4o temp=0.7 stream=True  
4.Embed: text-embedding-3-large 3072d batch=100  
5.Chunk: tiktoken 512tok 50overlap  
6.Worker: blob->extract->chunk->embed->pgvector->index  
7.RAG: cosine top-5->context->LLM  
8.tenacity 3x, Anthropic fallback  
9.VERIFY: embed, search, stream  
10.STATUS: AI=COMPLETE
```

AG-6 SECURITY

```
WAIT INFRA. 30m RED.  
1.NSG: 443 any, 22 admin-IP  
2.fail2ban: 3/3600  
3.SSH: key-only, no root  
4.Managed identity + Blob/Cognitive roles  
5..env chmod 600  
6.Azure Monitor CPU/disk/mem alerts  
7.UptimeRobot /api/health  
8.unattended-upgrades  
9.Lynis audit, fix HIGH  
10.VERIFY: nmap 443 only  
11.STATUS: SECURITY=COMPLETE
```

09 PREDICTIVE WALL CLOCK

Self-calibrating estimation engine. 5 inputs, auto-learns from actuals.

predicted = (base * complexity * zone) / (vm * agents * enhancements) * calibration

Variable	Values	Effect
VM	B2s=1.0 D2s=1.6 D4s=2.4 D8s=3.6	Divides time
Complexity	Trivial=0.3 Simple=0.6 Mod=1.0 Complex=1.8 Massive=3.2	Multiplies base
Agents	1-10. 1=1x 2=1.7x 4=2.8x 6=3.4x 10=4.2x	Divides time
Zone	G=1.0 Y=1.15 R=1.4	Multiplies (overhead)
Calibration	0.3-3.0x auto from last 20 actuals	Corrects bias

Enhancement	Factor
SKILLS	1.15x
ITERATION LOOPS	1.25x
SWARM COORD	1.30x
TEAM OF SWARMS	1.50x
SELF-HEAL	1.10x
PRE-CHECK	1.20x

```
# CALIBRATION: weighted avg of last 20 tasks
for i, task in enumerate(recent[-20:]):
    weight = 1 + i * 0.3 # recent heavier
    ratio = task.actual / task.predicted
    calibration = clamp(weighted_avg, 0.3, 3.0)
```

History	Band	Confidence
0-2	40%	LOW
3-9	25%	MEDIUM
10+	15%	HIGH

Interactive Wall Clock delivered as separate HTML artifact with all controls, timer, calibration, persistent history.

10 SETUP SCRIPT

```
#!/bin/bash
set -euo pipefail
DOMAIN='menagerie.yourdomain.com'
DB_PASS=$(openssl rand -base64 32)

# [1/8] SYSTEM
sudo apt update && sudo apt upgrade -y
sudo apt install -y nginx postgresql-16 postgresql-16-pgvector \
redis-server python3.12 python3.12-venv certbot \
python3-certbot-nginx fail2ban ufw logrotate curl

# [2/8] POSTGRES
sudo -u postgres psql -c "CREATE DATABASE menagerie;"
sudo -u postgres psql -c "CREATE USER menagerie_app PASSWORD '$DB_PASS';"
sudo -u postgres psql -d menagerie -c 'CREATE EXTENSION vector;'
# Tune: max_connections=200, shared_buffers=4GB

# [3/8] REDIS: maxmemory 2gb, allkeys-lru

# [4/8] PYTHON
python3.12 -m venv /opt/menagerie/venv
pip install fastapi 'uvicorn[standard]' asyncpg redis \
azure-storage-blob azure-search-documents openai tiktoken tenacity

# [5/8] NGINX+TLS: HTTP/2, gzip6, certbot --nginx

# [6/8] UFW: 443+22, deny all else

# [7/8] FAIL2BAN: sshd maxretry=3 bantime=3600

# [8/8] SYSTEMD: menagerie-api.service uvicorn x4 Restart=always

echo 'COMPLETE. DB Pass: $DB_PASS -- save to .env'
```

11 CLAUDE.md DROP-IN

```
# MENAGERIE VM - CLAUDE CODE RULES

## Architecture
- D4s_v5 Ubuntu24 East US. /opt/menagerie/
- Nginx(H2,gzip)->FastAPI(uvx4)->PG16+Redis
- External: Blob(SAS), AI Search, OpenAI(SSE)
- Deploy: ssh->git pull->systemctl restart

## File Ownership (NEVER cross)
- nginx->AG1 | schema->AG2 | app/->AG3
- services/storage->AG4 | services/ai->AG5 | .env+fw->AG6

## Zones: RED=human YELLOW=test GREEN=auto

## Time: 1m|5m|10m|30m|1hr|>2hr=WARNING

## Wall Clock (EVERY task)
- predicted=(base*complex*zone)/(vm*agents*enh)*cal
- Report: zone, impact[ABCDE], time, probability%

## Merge: A:Deploy B:Revenue C:Cost D:Org E:Legal

## Speed: H2, gzip6, request pool, Redis 5m TTL,
PgBouncer, SAS direct, SSE (not WebSocket)

## Errors: Restart=always, backup nightly, 2gb LRU,
certbot renew, fail2ban, stale job >10m=alert
```

12 NGINX CONFIG

```
upstream menagerie_api { server 127.0.0.1:8000; keepalive 32; }
limit_req_zone $binary_remote_addr zone=api:10m rate=100r/s;
server { listen 80; return 301 https://$server_name$request_uri; }
server {
    listen 443 ssl http2;
    ssl_protocols TLSv1.3; ssl_session_cache shared:SSL:10m;
    gzip on; gzip_vary on; gzip_proxied any;
    gzip_comp_level 6; gzip_min_length 256;
    gzip_types application/json text/plain text/css application/javascript;
    add_header X-Frame-Options DENY always;
    add_header Strict-Transport-Security 'max-age=31536000' always;
    add_header Access-Control-Allow-Origin 'tauri://localhost' always;
    add_header Access-Control-Allow-Methods 'GET,POST,PUT,DELETE,OPTIONS';
    add_header Access-Control-Allow-Headers 'Authorization,Content-Type';
    if ($request_method = 'OPTIONS') { return 204; }
    client_max_body_size 10m;
    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://menagerie_api;
        proxy_http_version 1.1; proxy_set_header Connection '';
        proxy_buffering off; proxy_cache off; proxy_read_timeout 300s;
    }
}
```

13 FASTAPI SKELETON

```
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
from contextlib import asynccontextmanager
import asyncpg, redis.asyncio as aioredis, time, json, os

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.pg = await asyncpg.create_pool(os.environ['DATABASE_URL'], min_size=5, max_size=20)
    app.state.redis = aioredis.from_url(os.environ['REDIS_URL'], decode_responses=True)
    yield; await app.state.pg.close(); await app.state.redis.close()

app = FastAPI(lifespan=lifespan)

@app.middleware('http')
async def log(req, call_next):
    t=time.perf_counter(); r=await call_next(req)
    print(f'{req.method} {req.url.path} {r.status_code} {(time.perf_counter()-t)*1000:.0f}ms')
    return r

@app.get('/api/health')
async def health(): return {'status':'ok'}

@app.post('/api/crud/{table}')
async def create(table, request: Request): pass # validate, INSERT, invalidate cache

@app.get('/api/crud/{table}/{rid}')
async def read(table, rid, request: Request): pass # Redis->PG fallback->cache

@app.post('/api/upload/sas')
async def sas(request: Request): pass # generate SAS, return url

@app.post('/api/upload/confirm')
async def confirm(request: Request): pass # metadata->PG, async vectorize

@app.post('/api/search')
async def search(request: Request): pass # AI Search hybrid query

@app.post('/api/chat')
async def chat(request: Request):
    async def gen():
        async for tok in ai.stream(data): yield f'data:{json.dumps({"t":tok})}\n\n'
    yield 'data:[DONE]\n\n'
    return StreamingResponse(gen(), media_type='text/event-stream')

@app.get('/api/jobs/{jid}')
async def job(jid): pass # query status table
```


14 SYSTEMD + BACKUP

menagerie-api.service

```
[Unit] Description=Menagerie FastAPI
After=network.target postgresql.service redis.service
[Service] Type=exec User=alan WorkingDirectory=/opt/menagerie
EnvironmentFile=/opt/menagerie/.env
ExecStart=/opt/menagerie/venv/bin/uvicorn app.main:app \
--host 127.0.0.1 --port 8000 --workers 4
Restart=always RestartSec=3
[Install] WantedBy=multi-user.target
```

backup.sh (cron 0 3 * * *)

```
#!/bin/bash
set -euo pipefail; TS=$(date +%Y%m%d_%H%M%S)
pg_dump -Fc menagerie | gzip > /tmp/m_$(TS).sql.gz
az storage blob upload --account-name menageriesa36965 \
--container menagerie-backups --name db_$(TS).sql.gz \
--file /tmp/m_$(TS).sql.gz --auth-mode login
rm /tmp/m_$(TS).sql.gz
```

15 COST ANALYSIS

Service	PAYG	1yr	3yr
VM D4s_v5	\$140	\$85	\$55
Disk 128GB	\$20	\$20	\$20
Blob 100GB	\$10	\$10	\$10
AI Search	\$75	\$75	\$75
OpenAI	~\$50	~\$50	~\$50
DNS+TLS	\$1	\$1	\$1
TOTAL/mo	\$296	\$241	\$211
TOTAL/yr	\$3,552	\$2,892	\$2,532

16 TIMELINE

Phase 1 (0:00-0:30): AG-1 provisions VM, installs, configures Nginx+TLS

Phase 2 (0:30-1:30): AG-2 thru AG-6 run in parallel. DB, API, Storage, AI, Security.

Phase 3 (1:30-2:00): End-to-end test, reboot verify, nmap audit.

Phase	Agents	Duration	Probability
1: Foundation	AG-1 solo	30 min	95%
2: Parallel	AG-2,3,4,5,6	60 min	85%
3: Verify	ALL	30 min	90%
TOTAL	ALL 6	2 hours	85%

TOTAL: 2 HOURS. Risk: Azure networking. Mitigation: AG-6 handles in parallel.

17 POST-DEPLOY CHECKLIST

#	Check	Pri
01	curl /api/health = 200	CRIT
02	HTTP/2 (h2) in headers	HIGH
03	gzip in headers	HIGH
04	PG tables = 51+	CRIT
05	redis-cli ping = PONG	CRIT
06	CRUD create+read	CRIT
07	CRUD update+delete	HIGH
08	SAS URL generated	CRIT
09	File uploads to Blob	CRIT
10	Confirm triggers job	HIGH
11	Search returns results	HIGH
12	Chat SSE TTFT <500ms	CRIT
13	PDF vectorize->search	HIGH
14	nmap: only 443	CRIT
15	Non-admin SSH blocked	CRIT
16	Reboot: auto-start <60s	CRIT
17	Backup runs, blob exists	HIGH
18	UptimeRobot active	MED

ALL CRITICAL PASS = PRODUCTION LIVE.

END. Section 11 -> Claude Code. Section 08 -> execute swarm. Wall Clock HTML -> track. Ship in 2 hours.